

패킷 추적 파일을 통한 네트워크 이벤트 분석 보고서

정하은

1. 서론

1.1. 과제 개요

본 보고서는 "패킷 추적 파일을 통한 네트워크 이벤트 분석"에 대한 내용을 담고 있다. 주어진 challenge.pcapng 추적 파일을 Wireshark를 기반으로 분석하여, 해당 파일 내에서 발생한 주요 네트워크 이벤트와 통신 흐름을 파악하는 것이 본 과제의 목표이다.

1.2. 문제 상황

특정 시간대에 수집된 challenge.pcapng 파일을 분석하여 파일 내의 패킷 수, TCP/HTTP 통신 흐름, 프로토콜 종류 등 핵심 네트워크 이벤트를 식별해야 했다.

2. 문제 분석 및 답변

2.1. 질문 1: 이 추적 파일에는 얼마나 많은 패킷이 있는가?

- 분석 : 제공된 텍스트 데이터의 마지막 번호와 Wireshark 화면 우측 하단에 표시된 패킷 수로 총 20개의 패킷이 있음을 확인했다. 이를 통해 전체 통신량의 대략적인 규모를 짐작할 수 있었다.
- 답변 : 이 추적 파일에는 총 20개의 패킷이 있다.

The image shows a Wireshark packet capture analysis of a file named 'challenge.pcapng'. The main packet list pane displays 20 packets, with the following details:

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	192.168.1.108	50.19.229.205	TCP	66	60139 → 80 [SYN] Seq=0 Win=8192 Len=0 MSS=1460 WS=4 SACK_PERM
2	0.092419	50.19.229.205	192.168.1.108	TCP	66	80 → 60139 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460 SACK_PERM WS=256
3	0.092521	192.168.1.108	50.19.229.205	TCP	54	60139 → 80 [ACK] Seq=1 Ack=1 Win=65700 Len=0
4	0.094027	192.168.1.108	50.19.229.205	HTTP	1384	GET /Tracking/V3/Instream/Impression/?start=287372147759041902826105[undefined]1338 3379 807 BBEEND&iari=148373...
5	0.196174	50.19.229.205	192.168.1.108	TCP	54	80 → 60139 [ACK] Seq=1 Ack=1331 Win=8960 Len=0
6	0.204299	50.19.229.205	192.168.1.108	HTTP	607	HTTP/1.1 302 Found
7	0.221142	192.168.1.108	50.19.229.205	HTTP	1380	GET /Tracking/V3/Instream/Impression/?start=287372147759041902826105[undefined]1338 3379 807 BBEEND&iari=148373...
8	0.326624	50.19.229.205	192.168.1.108	HTTP	607	HTTP/1.1 302 Found
9	0.608950	192.168.1.108	50.19.229.205	TCP	54	60139 → 80 [ACK] Seq=2657 Ack=1107 Win=64592 Len=0
10	0.674498	50.19.229.205	192.168.1.108	HTTP	607	[TCP Spurious Retransmission] HTTP/1.1 302 Found
11	0.674551	192.168.1.108	50.19.229.205	TCP	66	[TCP Dup ACK 9#1] 60139 → 80 [ACK] Seq=2657 Ack=1107 Win=64592 Len=0 SLE=554 SRE=1107
12	3.675382	192.168.1.108	50.19.229.205	HTTP	1428	GET /Tracking/V3/Instream/Impression/?start=287372147759041902826105[undefined]1338 3379 807 BBEEND&iari=148373...
13	3.776869	50.19.229.205	192.168.1.108	HTTP	607	HTTP/1.1 302 Found
14	3.975053	192.168.1.108	50.19.229.205	TCP	54	60139 → 80 [ACK] Seq=4031 Ack=1650 Win=65700 Len=0
15	11.175089	192.168.1.108	50.19.229.205	HTTP	1428	GET /Tracking/V3/Instream/Impression/?start=287372147759041902826105[undefined]1338 3379 807 BBEEND&iari=148373...
16	11.283954	50.19.229.205	192.168.1.108	HTTP	607	HTTP/1.1 302 Found
17	11.478274	192.168.1.108	50.19.229.205	TCP	54	60139 → 80 [ACK] Seq=5405 Ack=2213 Win=65144 Len=0

The bottom pane shows the packet details for the selected packet (No. 17):

- Frame 1: 66 bytes on wire (528 bits), 66 bytes captured (528 bits) on interface unknown, id 0
- Ethernet II, Src: 90:4c:e5:68:74:f6, Dst: 00:1d:7e:d9:94:c0
- Internet Protocol Version 4, Src: 192.168.1.108, Dst: 50.19.229.205
- Transmission Control Protocol, Src Port: 60139, Dst Port: 80, Seq: 0, Len: 0

The packet bytes pane shows the raw data in hexadecimal and ASCII.

2.2. 질문 2: IP 호스트는 무엇으로 프레임 1, 2, 3 안에서 TCP 연결을 만드는가?

• 분석

- 프레임 1: 192.168.1.108 → 50.19.229.205 [SYN] (클라이언트의 연결 요청)

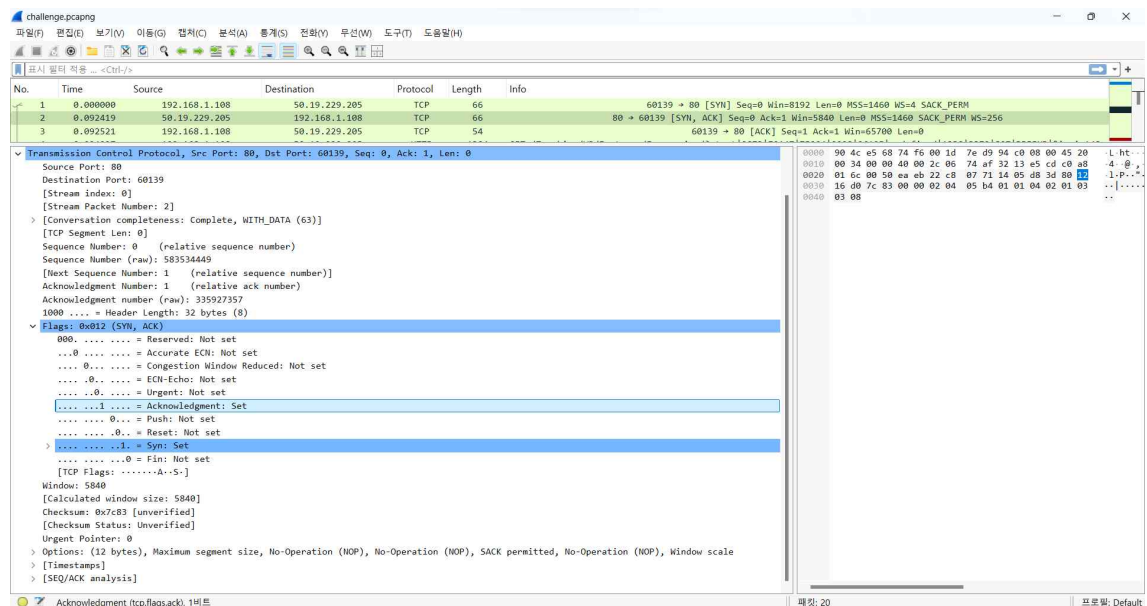
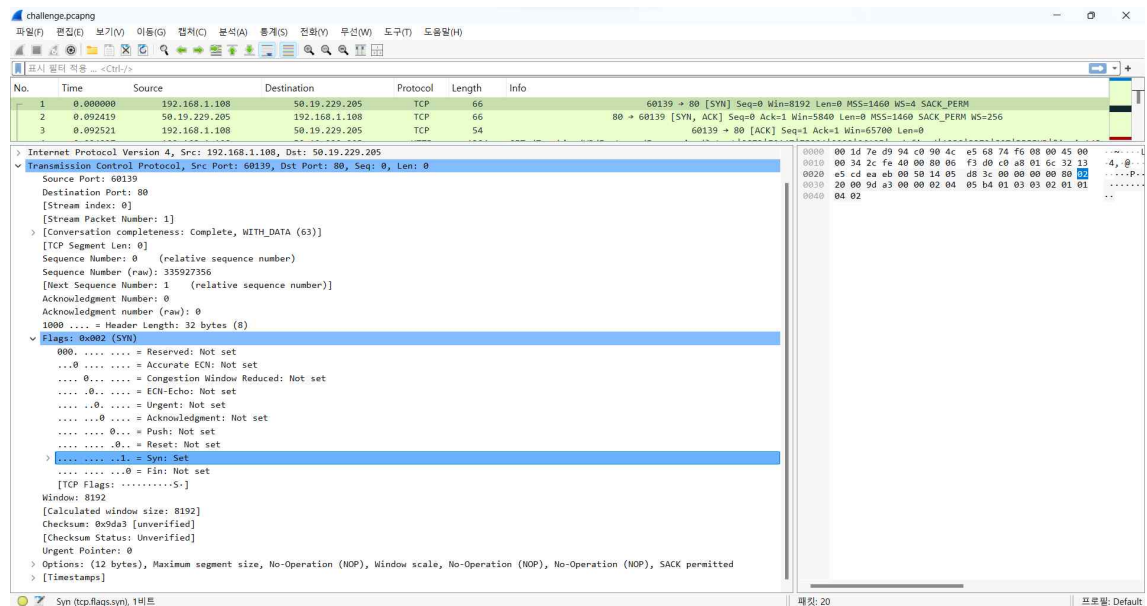
Source Port: 60139, Destination Port: 80

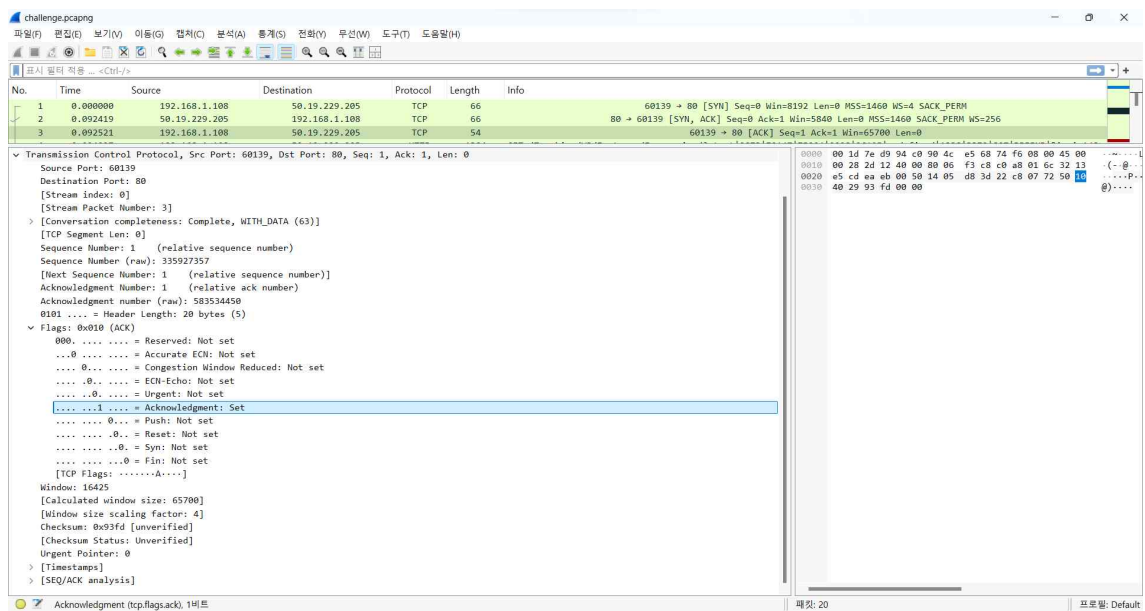
- 프레임 2: 50.19.229.205 → 192.168.1.108 [SYN, ACK] (서버의 응답)

Source Port: 80, Destination Port: 60139

- 프레임 3: 192.168.1.108 → 50.19.229.205 [ACK] (클라이언트의 최종 확인)

Source Port: 60139, Destination Port: 80





- **해석** : 분석 결과 192.168.1.108이 50.19.229.205로 향하는 TCP 연결을 시작했음을 알 수 있었다. Flag 필드를 확인하여 이 연결은 TCP의 **3-way handshake** 과정(SYN, SYN-ACK, ACK)을 통해 이루어졌음도 알 수 있었다. 특히, 192.168.1.108은 임시 포트(60139)를 사용하여 서버의 잘 알려진 HTTP 포트(80)로 연결을 시도했다. 이는 클라이언트가 웹 서버에 HTTP 통신을 위한 세션을 설정하는 과정이며, 안정적인 데이터 통신을 위한 기반을 마련한다.
- **답변** : 프레임 1, 2, 3에서 TCP 연결을 만드는 IP 호스트는 192.168.1.108이며, 이는 TCP **3-way handshake** 방식을 통해 Source Port 60139에서 Destination Port 80으로 연결을 수립했다.

2.3. 질문 3: 프레임 4로 보낸 HTTP 명령어는 무엇인가?

- 분석 : 프레임 4의 Packet Details 부분을 확인하여 GET HTTP 명령어를 사용했음을 확인했다. GET HTTP 명령어가 사용되었다는 것은 클라이언트(192.168.1.108)가 서버(50.19.229.205)로부터 특정 리소스(여기서는 /Tracking/V3/Instream/Impression/... 경로)를 요청하고 있음을 의미한다. 찾아보니 URL 경로(Tracking/V3/Instream/Impression)가 비디오 광고 서비스와 관련된 요청임을 알 수 있었다.

The image shows a Wireshark packet capture analysis. The top pane displays a list of packets. Packet 4 is highlighted, showing an HTTP GET request from 192.168.1.108 to 50.19.229.205. The middle pane shows the details of packet 4, including the Ethernet II header, Internet Protocol Version 4 header, and the Hypertext Transfer Protocol section. The bottom pane shows the raw packet data in hexadecimal and ASCII.

No.	Time	Source	Destination	Protocol	Length	Info
2	0.092419	50.19.229.205	192.168.1.108	TCP	66	80 → 60139 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460 SACK_PERM WS=256
3	0.092521	192.168.1.108	50.19.229.205	TCP	54	60139 → 80 [ACK] Seq=1 Ack=1 Win=65700 Len=0
4	0.094827	192.168.1.108	50.19.229.205	HTTP	1384	GET /Tracking/V3/Instream/Impression/?start[2873]72147[75904]9028[26105]undefined[1338]3379[807]BBEEND&iari=1483738cb=634687327792649999&in...
5	0.196174	50.19.229.205	192.168.1.108	TCP	54	80 → 60139 [ACK] Seq=1 Ack=1331 Win=49560 Len=0
6	0.204299	50.19.229.205	192.168.1.108	HTTP	607	HTTP/1.1 302 Found
7	0.221142	192.168.1.108	50.19.229.205	HTTP	1380	GET /Tracking/V3/Instream/Impression/?start[2873]72147[75904]9028[26105]undefined[1338]3379[807]BBEEND&iari=1483738cb=634687327792649999&in...
8	0.326624	50.19.229.205	192.168.1.108	HTTP	607	HTTP/1.1 302 Found
9	0.680950	192.168.1.108	50.19.229.205	TCP	54	60139 → 80 [ACK] Seq=2657 Ack=1107 Win=64592 Len=0

Frame 4: 1384 bytes on wire (11072 bits), 1384 bytes captured (11072 bits) on interface unknown, id 0

Ethernet II, Src: 98:4c:e5:08:74:fb, Dst: 08:0d:7e:d9:94:c8

Internet Protocol Version 4, Src: 192.168.1.108, Dst: 50.19.229.205

Transmission Control Protocol, Src Port: 60139, Dst Port: 80, Seq: 1, Ack: 1, Len: 1330

Hypertext Transfer Protocol

Request Method: GET

Request URL: /Tracking/V3/Instream/Impression/?start[2873]72147[75904]9028[26105]undefined[1338]3379[807]BBEEND&iari=1483738cb=634687327792649999&in...

Request URI Path: /Tracking/V3/Instream/Impression/

Request URI Query: start[2873]72147[75904]9028[26105]undefined[1338]3379[807]BBEEND&iari=1483738cb=634687327792649999&internalRedirect...

Request Version: HTTP/1.1

Accept: */*

Accept-Language: en-US

x-Flash-version: 11,1,102,55

Accept-Encoding: gzip, deflate

User-Agent: Mozilla/4.0 (compatible; MSIE 7.0; Windows NT 6.1; WOW64; Trident/5.0; SLCC2; .NET CLR 2.0.50727; .NET CLR 3.5.30729; .NET CLR 3...

Host: trk.vindicosuite.com

Connection: Keep-Alive

[...]Cookie: VINDICOMADIENCEISSUEDIDENTITY=f42e864f-a715-4be9-84bc-f24ff2abb3db; vpp=f42e864f-a715-4be9-84bc-f24ff2abb3db; __qca=P0-123841821...

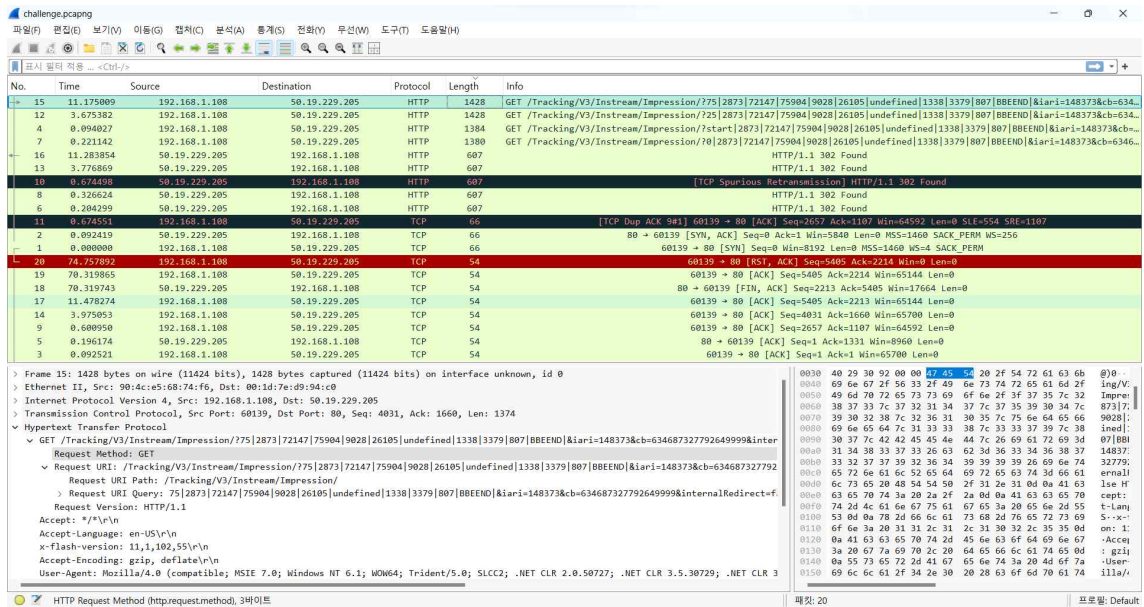
[...]Response in frame: 61

[...]Full request URI: http://trk.vindicosuite.com/Tracking/V3/Instream/Impression/?start[2873]72147[75904]9028[26105]undefined[1338]3379[807]BBEEND...

- 답변 : 프레임 4로 보낸 HTTP 명령어는 GET이다.

2.4. 질문 4: 이 추적 파일 안에서 가장 긴 프레임 길이는 얼마인가?

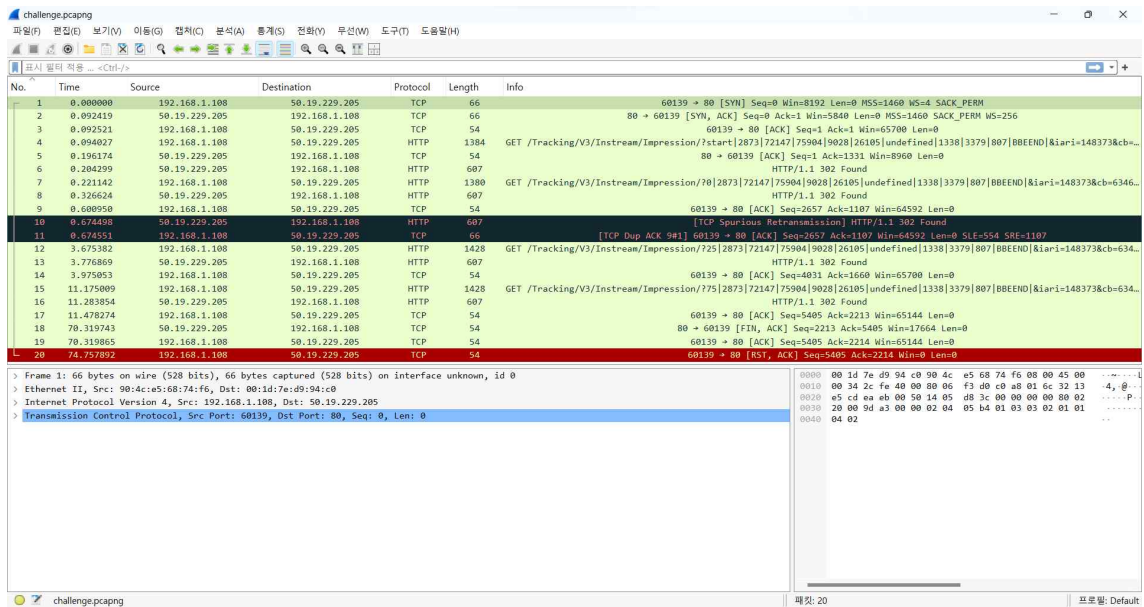
- 분석 : 각 프레임의 Length 필드를 내림차순으로 정렬하여 가장 큰 값을 찾을 수 있었다.
- 프레임 15: 1428 바이트
- 프레임 12: 1428 바이트
- 프레임 7: 1380 바이트
- 프레임 4: 1384 바이트
- 가장 긴 프레임 길이가 1428 바이트라는 것을 통해 해당 네트워크의 MTU(Maximum Transmission Unit)가 최소 1428 바이트 이상임을 예상할 수 있다. 실제로 이더넷의 표준 MTU는 1500바이트이므로, 이 길이는 정상적인 범위 내에 있다고 볼 수 있다.



- 답변 : 이 추적 파일 안에서 가장 긴 프레임 길이는 1428 바이트이다.

2.5. 질문 5: 어떤 프로토콜이 protocol 열에 보이는가?

- 분석 : 제공된 패킷 목록의 네 번째 열인 Protocol 열에 나타나는 프로토콜 이름들을 확인했다. 캡처 파일에서 TCP와 HTTP 프로토콜이 주로 보였으며, TCP는 데이터의 신뢰성 있는 전송을 보장하는 전송 계층 프로토콜이고, HTTP는 웹 콘텐츠 교환에 사용되는 애플리케이션 계층 프로토콜이다. 이는 클라이언트-서버 웹 서비스가 작동하고 있음을 나타낸다.



- 답변 : Protocol 열에 보이는 프로토콜은 주로 TCP와 HTTP이다.

2.6. 질문 6: HTTP 서버가 보낸 응답은 무엇인가?

- 분석 : HTTP 요청(GET 명령어)에 대한 서버(50.19.229.205)의 응답 패킷을 찾아 HTTP/1.1 뒤에 오는 상태 코드와 메시지를 확인했다.
 - 프레임 6: HTTP/1.1 302 Found
 - 프레임 8: HTTP/1.1 302 Found
 - 프레임 10: HTTP/1.1 302 Found [TCP Spurious Retransmission] 경고
 - 프레임 13: HTTP/1.1 302 Found
 - 프레임 16: HTTP/1.1 302 Found

The image shows a Wireshark packet capture of an HTTP transaction. The packet list on the left shows several packets, with packet 6 being the first 'HTTP/1.1 302 Found' response. The packet details pane on the right shows the structure of the HTTP response, including the status code 302 and the 'Found' message. The packet bytes pane on the right shows the raw data of the packet, including the HTTP status line and headers.

No.	Time	Source	Destination	Protocol	Length	Info
5	0.196174	50.19.229.205	192.168.1.108	TCP	54	80 → 60139 [ACK] Seq=1 Ack=1331 Win=8960 Len=0
6	0.204299	50.19.229.205	192.168.1.108	HTTP	607	HTTP/1.1 302 Found
7	0.221142	192.168.1.108	50.19.229.205	HTTP	1380	GET /Tracking/V3/Instream/Impression/70[2873]72147[75904]9028[26105]undefined[1338]3379[807]BBEEND[&iar=1483738...
8	0.326624	50.19.229.205	192.168.1.108	HTTP	607	HTTP/1.1 302 Found
9	0.508050	192.168.1.108	50.19.229.205	TCP	54	60139 → 80 [ACK] Seq=2657 Ack=1107 Win=64592 Len=0
10	0.574498	50.19.229.205	192.168.1.108	HTTP	607	[TCP Spurious Retransmission] HTTP/1.1 302 Found
11	0.574551	192.168.1.108	50.19.229.205	TCP	66	[TCP Dup ACK 9#1] 60139 → 80 [ACK] Seq=2657 Ack=1107 Win=64592 Len=0 SLE=554 SRE=1107
12	3.675382	192.168.1.108	50.19.229.205	HTTP	1428	GET /Tracking/V3/Instream/Impression/725[2873]72147[75904]9028[26105]undefined[1338]3379[807]BBEEND[&iar=1483738...
13	3.776869	50.19.229.205	192.168.1.108	HTTP	607	HTTP/1.1 302 Found

Internet Protocol Version 4, Src: 50.19.229.205, Dst: 192.168.1.108
Transmission Control Protocol, Src Port: 80, Dst Port: 60139, Seq: 1, Ack: 1331, Len: 553
Hypertext Transfer Protocol
HTTP/1.1 302 Found
Response Version: HTTP/1.1
Status Code: 302
[Status Code Description: Found]
Response Phrase: Found
Cache-Control: no-cache
Date: Fri, 30 Mar 2012 23:32:59 GMT
Expires: -1
Location: /tracking/dot.gif
P3P: CP="NOI DSP COR DEVa TATA OUR BUS UNI NAV"
Pragma: no-cache
Server: Microsoft-IIS/7.5
Set-Cookie: f42e864f-a715-4be9-84bc-f24ff2abb3db_vpp=1333148250:InstreamImpression_Click:9028[807]2788[72022]26622[75898[1338]3379:1; domain=.aspnet-version: 4.0.30319
X-VINDICO-CHL: 784
X-VINDICO-Instance: i-11920872
Content-Length: 0
Connection: keep-alive
[request in frame: 4]
[Time since request: 0.110272000 seconds]
[Request URI: /Tracking/V3/Instream/Impression/?start[2873]72147[75904]9028[26105]undefined[1338]3379[807]BBEEND[&iar=1483738&cb=63468732779]
[Full request URI: http://trk.vindicosuite.com/Tracking/V3/Instream/Impression/?start[2873]72147[75904]9028[26105]undefined[1338]3379[807]BBEEND[&iar=1483738&cb=63468732779]

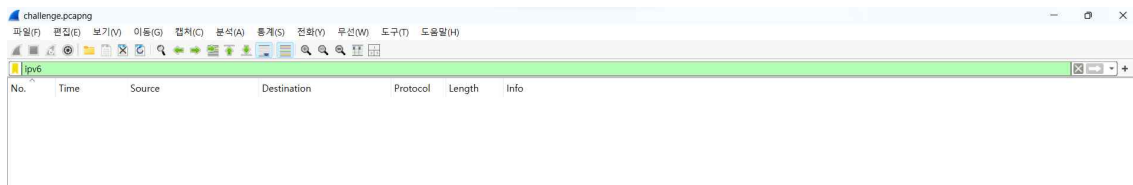
- 해석
 - HTTP 302 Found 응답 : 이 상태 코드는 서버가 요청된 리소스를 일시적으로 다른 URI에서 찾았음을 나타낸다. 즉, 서버는 클라이언트에게 현재 요청한 페이지/리소스 대신 Location 헤더에 명시된 다른 URL로 리다이렉션을 보내라고 지시하는 것이다. 이는 301 Moved Permanently(영구 이동)와 달리 일시적인 변경을 의미하며, 주로 서버 유지보수, A/B 테스트, 또는 특정 조건에 따른 동적 페이지 이동 등에 사용된다고 한다.
 - 반복적인 302 응답 : 캡처 파일에서 여러 번의 GET 요청마다 반복적으로 302 Found 응답이 나타나는 것은, 클라이언트가 특정 기능을 수행하기 위해 여러 단계의 리다이렉션을 거쳐야 하는 상황임을 뜻한다. 특히 URL 경로에 "Tracking"이 포함되어 있는 것으로 보아, 이는 온라인 광고 추적 시스템에서 흔히 발생하는 시나리오라고 생각된다. 사용자가 광고를 클릭하면, 여러 광고 플랫폼이나 측정 시스템을 거치며 연속적인 리다이렉션이 발생하고, 최종적으로 목표 페이지에 도달하게 되는 과정을 말한다. 이러한 다단계 리다이렉션은 광고 성과 측정, 사용자 행동 분석 등을 위해 사용된다.
 - 프레임 10의 [TCP Spurious Retransmission] : 이 표시는 "가짜 재전송" 또는 "불필요한 재전송"을 의미한다. TCP는 데이터 손실을 감지하면 해당 데이터를 재전송하는데, Spurious Retransmission은 실제로는 데이터 손실이 발생하지 않았거나, 이미 수신 측에서 해당 데이터

의 ACK를 보냈음에도 불구하고, 송신 측에서 타이머 만료 등으로 인해 데이터를 다시 보낸 경우에 Wireshark가 표시하는 경고이다.

- 이는 일반적으로 네트워크 지연(Latency), 패킷 순서 바뀜(Out-of-Order delivery), 또는 송수신 버퍼 크기 문제 등으로 인해 발생할 수 있다. 즉, 송신 측(50.19.229.205)이 수신 측(192.168.1.108)으로부터 제때 ACK를 받지 못해 재전송을 결정했지만, 실제로는 ACK가 도중에 지연되거나 패킷 순서가 바뀌어 도착했을 가능성이 있다. 이는 네트워크의 안정성이나 성능 문제를 간접적으로 시사할 수 있으며, 특히 웹 서비스의 경우 사용자 경험에 미미한 지연을 줄 수 있다.
- **답변** : HTTP 서버(50.19.229.205)가 보낸 응답은 HTTP/1.1 302 Found 이다. 이 응답은 클라이언트에게 리소스를 다른 위치에서 찾았으니 **새로운 URL로 다시 요청(리다이렉션)**하라는 지시이다. 이 추적 파일에서는 여러 번의 HTTP 요청에 대해 이 302 응답이 반복적으로 나타나며, 이는 다단계의 리다이렉션 체인이 작동하고 있음을 추측할 수 있다. 또한, 프레임 10에서 보이는 **[TCP Spurious Retransmission]**은 서버가 데이터를 불필요하게 재전송했음을 나타내며, 이는 네트워크 지연이나 패킷 순서 변경 등 작은 네트워크 문제의 징후일 수 있다.

2.7. 질문 7: 이 추적 파일 안에 IPv6 트래픽이 있는가?

- **분석** : 패킷 필터링(IPv6)을 통해 IPv6 트래픽이 존재하는지 확인해보았는데 빈 화면으로 나왔다. 이 캡처 파일에 IPv6 트래픽이 없다는 것은 통신이 전부 **IPv4** 환경에서 이루어졌음을 아 나타낸다. 이를 통해 해당 네트워크 환경이 IPv4만 사용하거나, 적어도 캡처된 파일 내에서는 IPv6 통신이 발생하지 않았음을 알 수 있다. 여전히 많은 시스템이나 특정 네트워크에서는 **IPv4**만 사용하는 경우가 많다는 사실도 엿볼 수 있었다.



- **답변** : 이 추적 파일 안에 IPv6 트래픽은 없다. 모든 트래픽은 **IPv4**를 사용하고 있다.

2.8. 추가 분석: TCP 연결 종료 과정

- **분석**
 - 프레임 18: 50.19.229.205 → 192.168.1.108 [FIN, ACK] Seq=2213 Ack=5405
 - 프레임 19: 192.168.1.108 → 50.19.229.205 [ACK] Seq=5405 Ack=2214
 - 프레임 20: 192.168.1.108 → 50.19.229.205 [RST, ACK] Seq=5405 Ack=2214
 - 파일의 마지막 패킷들은 TCP 연결 종료 과정을 보여준다.
- **해석** : 프레임 18에서 TCP 연결 종료는 서버가 먼저 FIN 플래그를 보내 시작되었지만, 클라이언트는 이에 대한 **ACK**를 보낸 후 정상적인 종료 대신 **RST** 플래그로 연결을 강제 종료했다. 이는 클라이언트 측에서의 **4-way handshake** 과정을 거치지 않고 비정상적인 연결 종료를 했음을 의미한다.

2.9. Flow Graph



3. 결론

challenge.pcapng 파일을 분석한 결과, 해당 패킷 캡처는 주로 TCP 및 HTTP 프로토콜을 사용한 웹 통신 이벤트의 패킷을 수집한 것임을 확인할 수 있었다. 클라이언트(192.168.1.108)는 서버(50.19.229.205)와 TCP 연결을 성공적으로 수립한 후, /Tracking/V3/Instream/Impression 경로에 대한 HTTP GET 요청을 반복적으로 전송했다. 이에 대해 서버는 일관되게 HTTP/1.1 302 Found라는 리다이렉션 응답을 반환했다. 이는 광고 추적과 같이 다단계의 웹 리다이렉션이 필요한 시나리오에서 흔히 볼 수 있는 통신 패턴이다. 파일 내 가장 긴 프레임은 1428 바이트로 일반적인 이더넷 환경의 MTU에 부합하며, 모든 트래픽은 IPv4 기반으로 이루어져 IPv6 트래픽은 발견되지 않았다. 더불어, TCP 연결 종료 과정에서는 서버가 먼저 FIN 플래그를 보내 연결 종료를 요청했으나, 클라이언트가 이에 대한 ACK를 보낸 후 정상적인 FIN 응답 대신 RST 플래그를 보내 연결을 강제로 종료했음을 확인했다. 이는 클라이언트 측에서의 비정상적인 연결 종료를 시사한다.

이러한 분석을 통해 특정 시간대의 네트워크 통신 흐름과 주요 웹 상호작용 이벤트, 그리고 연결 종료 시 발생한 비정상적인 패턴까지 명확히 이해할 수 있었다.